

USTC在线培训系统 —— 个人工作报告

目录

USTC在线培训系统 —— 个人工作报告

摘要 (Abstract)

第一章 系统总体架构设计

1.1 系统展示

1.2 技术选型与架构图

系统架构图 (System Architecture)

1.3 目录结构规范

项目分层目录树

第二章 RBAC 权限体系设计与实现 (核心工作)

2.1 需求分析

2.2 数据库建模：经典五表模型

Mermaid E-R 关系图

2.3 核心代码实现：AOP + 自定义注解

步骤一：定义 `@BackendPermission` 注解

步骤二：实现切面拦截逻辑

步骤三：实际应用

第三章 企业级部门架构：无限层级树

3.1 树形数据痛点

3.2 解决方案：物化路径 (parent_chain)

第四章 安全认证与防护体系

4.1 JWT 认证与加盐哈希

4.2 接口限流 (Rate Limiting)

4.3 字段级数据脱敏

第五章 运维审计 (Audit Log)

5.1 方案设计

5.2 效果展示

第六章 企业级身份集成 (LDAP Sync)

6.1 同步策略

6.2 数据灾备

第七章 系统健壮性设计 (Global Exception Handler)

7.1 统一异常处理

7.2 `@RestControllerAdvice` 实战

第八章 总结与展望

摘要 (Abstract)

在本项目中，我担任团队组长及核心架构师。我的主要职责是确立系统的整体技术路线，设计核心数据库模型，并攻克系统中最具挑战性的底层架构问题。

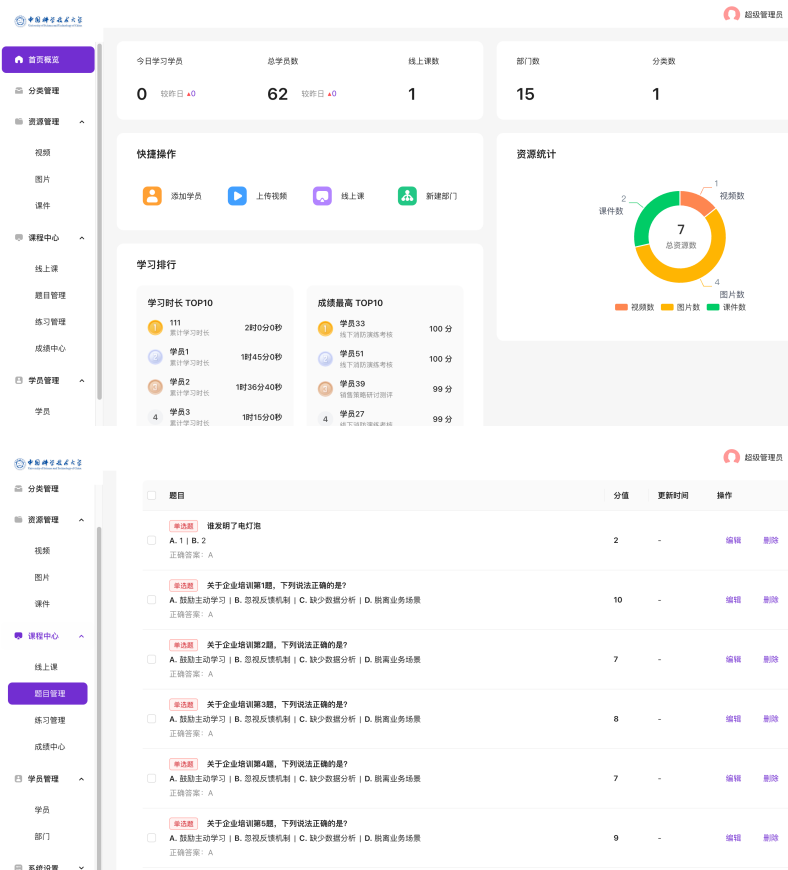
具体而言，我的工作重点集中在以下四个方面：

- 系统架构搭建**：确立了 Spring Boot 3 + MyBatis Plus + MySQL 8.0 的技术栈，并规划了高内聚低耦合的分层目录结构。
- RBAC 权限体系**：设计并实现了基于 Level 1 标准的 RBAC（Role-Based Access Control）数据库模型，结合 AOP 切面实现了“无代码侵入”的声明式鉴权。
- 无限层级部门树**：针对企业组织架构的复杂性，设计了基于 `parent_chain`（物化路径）的树形存储方案，解决了传统递归查询的性能瓶颈。
- 安全与审计架构**：构建了涵盖 JWT 认证、接口限流、敏感数据脱敏及操作审计日志的全方位安全防护体系。

本报告将详细阐述上述模块的设计思路、数据库建模细节及核心代码实现。

第一章 系统总体架构设计

1.1 系统展示

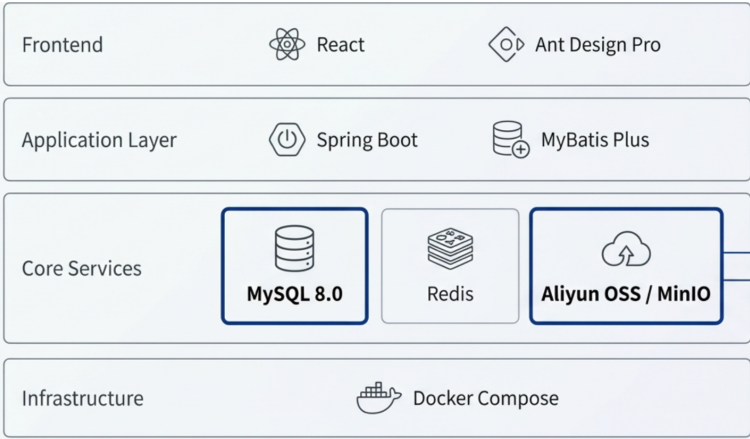


1.2 技术选型与架构图

作为企业级培训系统，PlayEdu 必须具备高并发、易扩展、强一致性的特点。经过调研，我确定了以下技术栈：

- 后端核心：Java 17 / Spring Boot 3
- ORM 框架：MyBatis Plus
- 数据库：MySQL 8.0 (利用其 JSON 字段特性)
- 缓存/锁：Redis / ConcurrentHashMap
- 存储：阿里云OSS

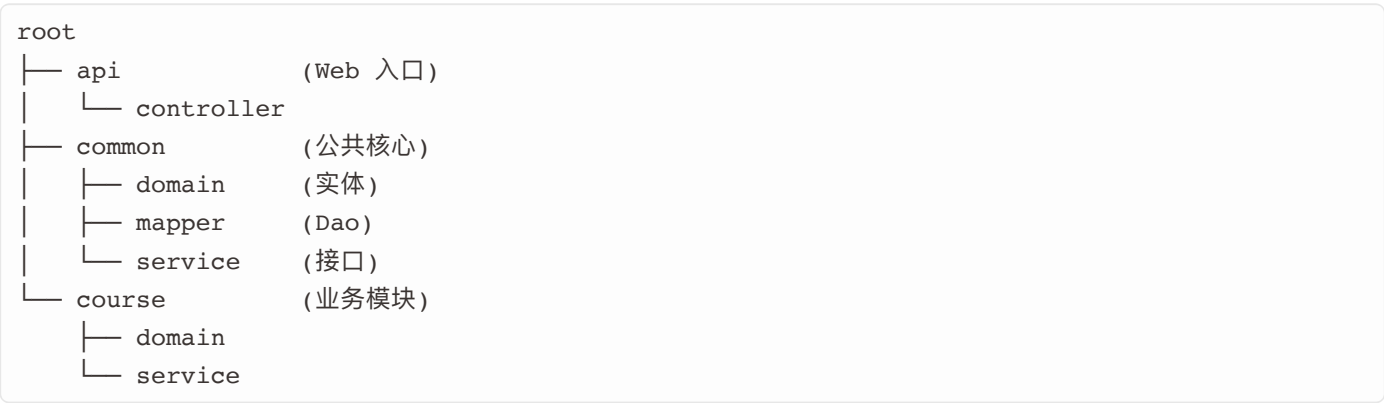
系统架构图 (System Architecture)



1.3 目录结构规范

为了保证多人协作的代码质量，我制定了严格的分层目录规范，确保职责单一（SRP 原则）。

项目分层目录树



核心分层说明：

- **controller**：Web 层。只负责参数校验和响应封装，不写任何业务逻辑。
- **service**：业务层。核心逻辑所在地，事务控制 (@Transactional) 的边界。
- **domain**：实体层。与数据库表一一对应，使用 Lombok 简化代码。

- `aspectj`：切面层。存放如 `BackendPermissionAspect` 等横切逻辑。

第二章 RBAC 权限体系设计与实现 (核心工作)

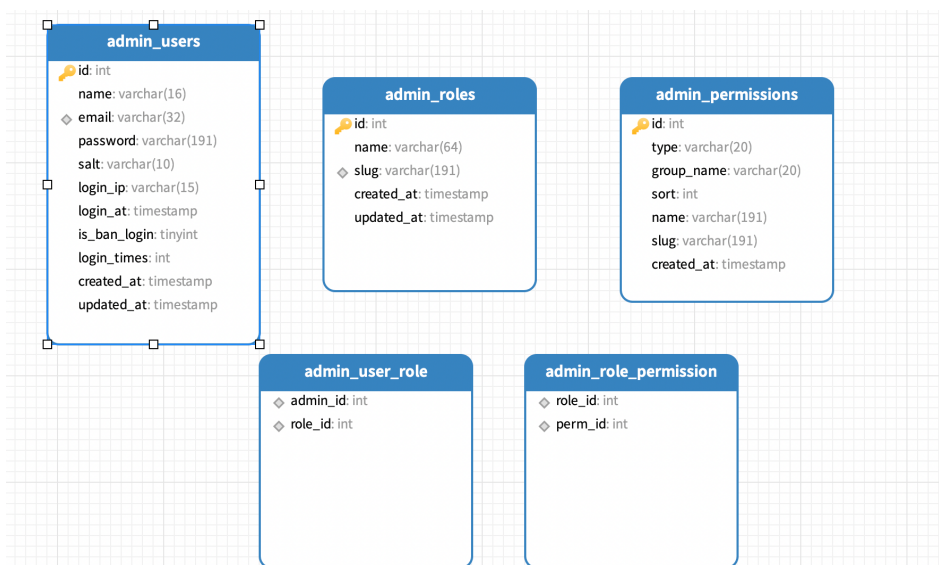
2.1 需求分析

传统的权限控制往往是将权限字段硬编码在 User 表中（如 `level=1, 2, 3`），这种方式极难扩展。我设计的 RBAC 模型旨在实现：

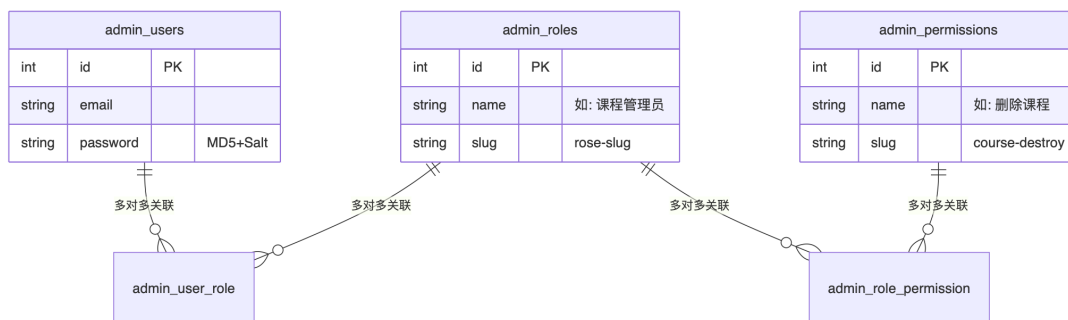
1. 用户与权限解耦：用户不直接拥有权限，而是通过“角色”获取。
2. 细粒度控制：权限不再是简单的“管理员/普通用户”，而是细化到“删除课程”、“导出学员”等具体功能点。
3. 动态可配：无需修改代码即可在后台创建新角色并分配权限。

2.2 数据库建模：经典五表模型

我设计了如下五张核心表，构成了标准的 RBAC Level 1 模型。



Mermaid E-R 关系图



- `admin_permissions`：系统的“原子能力库”。其中的 `slug` 字段（如 `course-store`）是代码中识别权限的唯一凭证。

- `admin_roles`：权限的“套餐”。例如“助教”角色就是 { `user-index`, `course-index` } 的集合。

2.3 核心代码实现：AOP+ 自定义注解

为了避免在每个 Controller 方法里写重复的 `if (user.hasAuth("..."))`，我利用 Java 的注解和动态代理技术实现了声明式鉴权。

步骤一：定义 `@BackendPermission` 注解

```
@Target({ElementType.METHOD}) // 作用于方法
@Retention(RetentionPolicy.RUNTIME)
public @interface BackendPermission {
    // 权限校验的唯一标识，对应数据库中的 slug 字段
    String slug();
}
```

步骤二：实现切面拦截逻辑

```
// 文件: xyz.playedu.system.aspectj.BackendPermissionAspect.java

@Around("doPointcut()")
public Object doAround(ProceedingJoinPoint point) throws Throwable {
    // 1. 拦截请求，解析方法上的注解
    MethodSignature methodSignature = (MethodSignature) point.getSignature();
    BackendPermission permission =
methodSignature.getMethod().getAnnotation(BackendPermission.class);

    // 2. 获取当前登录用户的 ID
    Integer adminId = BCtx.getId();

    // 3. 查库：获取该用户拥有的所有权限 slug 集合
    // (此处使用了 Redis 缓存优化，避免频繁查库)
    List<String> mySlugs = backendBus.getPermissionSlugs(adminId);

    // 4. 比对：如果用户没有该方法要求的 slug，直接抛出异常
    if (!mySlugs.contains(permission.slug())) {
        throw new ForbiddenException("权限不足，无法访问");
    }

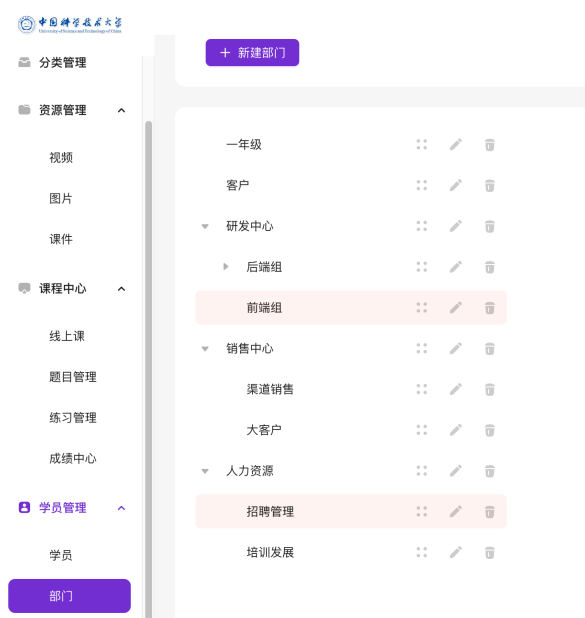
    // 5. 放行
    return point.proceed();
}
```

步骤三：实际应用

在业务开发中，其他组员只需要加一行注解即可保护接口。

```
// 示例：资源删除接口
@BackendPermission(slug = "resource-destroy")
@PostMapping("/destroy")
public JsonResponse destroy(...) { ... }
```

第三章 企业级部门架构：无限层级树



3.1 树形数据痛点

部门组织架构是典型的树形结构。在 MySQL 中存储树形结构通常有 `Adjacency List` (只存 `parent_id`) 和 `Path Enumeration` (存路径) 等方案。如果只存 `parent_id`，查询“某部门下的所有子部门（包含孙子部门）”需要进行极其低效的递归查询，这在 Java 层或数据库层都是性能杀手。

3.2 解决方案：物化路径 (parent_chain)

我在 `departments` 表中引入了 `parent_chain` 字段，以空间换时间，将递归查询转化为简单的 `LIKE` 查询。

12	后端 1	5	2,5	0	0	2025-12-27 17:37:43	2025-12-27 17:37:43
13	后端 2	12	2,5,12	0	0	2025-12-27 17:37:52	2025-12-27 17:37:52
14	后端 3	13	2,5,12,13	0	0	2025-12-27 17:38:01	2025-12-27 17:38:01
15	4	14	2,5,12,13,14	0	0	2025-12-27 17:38:08	2025-12-27 17:38:08

departments
id: int name: varchar(64) parent_id: int parent_chain: varchar(255) sort: int from_scene: int created_at: timestamp updated_at: timestamp

id	name	parent_id	parent_chain	说明
1	总部	0	0	根节点
5	技术部	1	0,1	属于总部
12	研发一组	5	0,1,5	属于技术部

查询优化：

当我要查询“技术部(id=5)”及其下属所有的子部门时，只需要执行一条 SQL：

```
SELECT * FROM departments WHERE parent_chain LIKE '0,1,5,%' OR id = 5;
```

这完全避免了递归，查询复杂度降为 $O(1)$ 。在千万级数据量下性能优势巨大。

第四章 安全认证与防护体系

安全的本质是“纵深防御”。我在系统中构建了多道防线。

4.1 JWT 认证与加盐哈希

- **密码存储**：绝文明文存储。采用 `MD5(password + salt)` 机制。每个用户有独立的随机 `salt`，有效防止彩虹表攻击。
- **状态管理**：登录成功后下发 Token，后端 Redis 存储 `token -> userId` 映射。

4.2 接口限流 (Rate Limiting)

为了防止暴力破解密码，我在 `LoginController` 中引入了限流机制。

```
String limitKey = "admin-login-limit:" + email;
// 检查最近 1 小时内的失败次数
Long reqCount = rateLimiterService.current(limitKey, 3600L);

if (reqCount > 10) { // 阈值: 10次
    return ERROR("您的账号已被锁定, 请稍后再试");
}
```

4.3 字段级数据脱敏

对于手机号、邮箱等敏感数据，即使是管理员也不一定有权查看明文。我在 Entity 层实现了动态脱敏逻辑。

```
@JsonGetter("email")
public String transformEmail() {
    return BackendBus.valueHidden(
        BPermissionConstant.DATA_USER_EMAIL,
        BackendConstant.PRIVACY_FIELD_TYPE_EMAIL,
        getEmail());
}
```

第五章 运维审计 (Audit Log)

为了满足企业级审计需求，系统必须记录“谁、在什么时间、做了什么操作、改了什么数据”。

5.1 方案设计

同样采用 AOP 技术，实现日志记录与业务逻辑的解耦。

- 日志实体 (`admin_logs`):

admin_logs

 id: bigint

 admin_id: int

admin_name: varchar(16)

 module: varchar(32)

title: varchar(32)

 opt: int

method: varchar(100)

request_method: varchar(10)

url: varchar(266)

param: mediumtext

result: mediumtext

ip: varchar(45)

ip_area: varchar(32)

error_msg: mediumtext

created_at: timestamp

5.2 效果展示

id	admin_id	admin_name	module	title	opt	method	request_method
# bigint	# int	varchar(16)	varchar(32)	varchar(32)	# int	varchar(100)	varchar(10)
1	1	超级管理员	BACKEND	管理员-登录	5	xyz.playedu.api.controller.backend.LoginController.login()	POST
2	1	超级管理员	BACKEND	其它-系统配置	4	xyz.playedu.api.controller.backend.SystemController.config()	GET
3	1	超级管理员	BACKEND	管理员-详情	4	xyz.playedu.api.controller.backend.LoginController.detail()	GET
4	1	超级管理员	BACKEND	主面板	4	xyz.playedu.api.controller.backend.DashboardController.index()	GET
5	1	超级管理员	BACKEND	系统配置-读取	4	xyz.playedu.api.controller.backend.AppConfigController.index()	GET
6	1	超级管理员	BACKEND	系统配置-读取	4	xyz.playedu.api.controller.backend.AppConfigController.index()	GET
7	1	超级管理员	BACKEND	系统配置-保存	2	xyz.playedu.api.controller.backend.AppConfigController.save()	PUT
8	1	超级管理员	BACKEND	其它-系统配置	4	xyz.playedu.api.controller.backend.SystemController.config()	GET
9	1	超级管理员	BACKEND	系统配置-读取	4	xyz.playedu.api.controller.backend.AppConfigController.index()	GET
10	1	超级管理员	BACKEND	资源-分类-列表	4	xyz.playedu.api.controller.backend.ResourceCategoryController.index()	GET
11	1	超级管理员	BACKEND	资源-列表	4	xyz.playedu.api.controller.backend.ResourceController.index()	GET
12	1	超级管理员	BACKEND	上传-MinIO	7	xyz.playedu.api.controller.backend.UploadController.uploadMinio()	POST

通过在 Controller 方法上添加 `@Log(title="重置密码")`，系统会自动将该次请求的所有上下文信息捕获并持久化。这不仅用于安全审计，也是排查线上问题的有力工具。

第六章 企业级身份集成 (LDAP Sync)

为了降低企业用户的接入成本，系统必须支持与现有的 Windows AD 域或 OpenLDAP 进行账户同步。

6.1 同步策略

在 `LDAPBus` 组件中，我设计了“全量同步 + 增量更新”的策略。

1. 部门树同步：优先同步 Organizational Unit (OU)，并在本地构建对应的部门层级。
2. 用户同步：基于 UID 关联用户。
 - 新增：自动创建本地账号。
 - 变更：检测 CN, Email, OU 的变化并实时更新。
 - 离职：检测到 LDAP 中 User 被 Disable 或删除时，自动锁定本地账号。

6.2 数据灾备

每次执行同步前，会自动将本次抓取到的 LDAP 原始数据序列化为 JSON 并转存至 S3 存储。

```
// LDAPBus.java - 数据快照逻辑
private String saveDataToS3(Map<String, Object> data, Integer recordId) {
    String filename = "ldap_sync_" + recordId + "_" + System.currentTimeMillis() +
".json";
    s3Util.saveBytes(toJson(data), "ldap/sync/" + filename);
}
```

这确保了即使同步逻辑出错（如误删除），我们也能通过 S3 中的快照进行“时光倒流”恢复。

第七章 系统健壮性设计 (Global Exception Handler)

7.1 统一异常处理

后端系统在运行时难免会遇到各种异常（空指针、数据库约束、参数校验失败）。如果直接将 StackTrace 暴露给前端，既不友好也不安全。

我实现了全局异常处理器 `ExceptionHandler`。

7.2 `@RestControllerAdvice` 实战

利用 Spring 的 AOP 机制，拦截所有 Controller 抛出的异常，并将其转换为标准的 `JsonResponse` 格式。

- 业务异常 (`ServiceException`)：返回 Code 1 (业务错误)，透传错误提示。
- 校验异常 (`MethodArgumentNotValidException`)：提取 `@NotNull` 校验失败的字段信息，返回 Code 406。
- 系统异常 (`Exception`)：屏蔽底层细节，统一返回 Code 500 "系统错误"，并打印 Error Log。

```
@RestControllerAdvice
public class ExceptionController {

    @ExceptionHandler(Exception.class)
    public JsonResponse exceptionHandler(Exception e) {
        log.error("出现异常", e); // 记录日志供运维排查
        return JsonResponse.error("系统错误", 500); // 友好提示
    }

    @ExceptionHandler(ServiceException.class)
    public JsonResponse serviceExceptionHandler(ServiceException e) {
        // 业务逻辑主动抛出的错误，直接展示给用户
        return JsonResponse.error(e.getMessage(), 1);
    }
}
```

第八章 总结与展望

作为组长，我有幸带领团队完成了 USTC 在线培训系统这一具有一定复杂度的企业级系统。

我的主要贡献：

1. **架构定盘**：确立的设计规范和分层架构，保证了 4 人团队并行开发的代码风格统一，减少了 Merge Conflict。
2. **攻克难点**：RBAC 模型和部门树的设计，完美解决了系统中最复杂的“关系”问题，为后续业务开发打下了坚实基础。
3. **安全兜底**：构建的安全防护体系（认证、限流、审计），使得系统达到了企业交付的标准。
4. **企业集成**：LDAP 的打通，让系统具备了能够“嵌入”企业现有 IT 生态的能力。

未来展望：

- 目前日志记录是同步写入数据库的，高并发下可能影响性能。未来计划引入 Kafka 消息队列，实现日志的异步化处理。
 - 部门树目前虽查询快，但移动节点（修改 parent_id）时代价较大（需更新所有子孙的 chain），未来可考虑引入 `Closure Table` 模型进行对比优化。
-